

Contents

1	Note on Lab 8	2
2	Exploring and preparing the dataset	2
3	Training and evaluating the model	3
4	Logistic regression model	5
4.1	Logistic regression with scaled data	7
4.2	Scaling separately	8

1 Note on Lab 8

This lab solution only involves the use of the code taught in the tutorial, i.e. with the `knn()` function from `class`. Again, I would emphasise and encourage you to use the `train()` function from `caret`, which is much more robust in the training process. It also simplifies your training process.

Again, take note on the use of the `preProcess` parameter in the `train()` function, if you have already normalised your data.

2 Exploring and preparing the dataset

The data is stored in a `.csv` file, which we read in. We can see that there are 400 observations of 4 variables, and `admit` is our outcome variable of interest. The scales of all the features are also extremely different.

```
grad <- read.csv('gradadmit.csv', header=T)
str(grad)
```

```
## 'data.frame':    400 obs. of  4 variables:
## $ admit: int  0 1 1 1 0 1 1 0 1 0 ...
## $ gre  : int  380 660 800 640 520 760 560 400 540 700 ...
## $ gpa  : num  3.61 3.67 4 3.19 2.93 3 2.98 3.08 3.39 3.92 ...
## $ rank : int  3 3 1 4 4 2 1 2 3 2 ...
```

```
summary(grad)
```

##	admit	gre	gpa	rank
## Min.	:0.0000	Min. :220.0	Min. :2.260	Min. :1.000
## 1st Qu.:	:0.0000	1st Qu.:520.0	1st Qu.:3.130	1st Qu.:2.000
## Median :	:0.0000	Median :580.0	Median :3.395	Median :2.000
## Mean	:0.3175	Mean :587.7	Mean :3.390	Mean :2.485
## 3rd Qu.:	:1.0000	3rd Qu.:660.0	3rd Qu.:3.670	3rd Qu.:3.000
## Max.	:1.0000	Max. :800.0	Max. :4.000	Max. :4.000

We convert `admit` into a factor and perform a min-max normalisation on the predictors.

```
grad$admit <- factor(grad$admit)

nor <- function(x) { (x - min(x)) / (max(x) - min(x)) }
grad.nor <- grad
grad.nor[2:4] <- sapply(grad.nor[2:4], nor)
str(grad.nor)
```

```
## 'data.frame':    400 obs. of  4 variables:
##  $ admit: Factor w/ 2 levels "0","1": 1 2 2 2 1 2 2 1 2 1 ...
##  $ gre  : num  0.276 0.759 1 0.724 0.517 ...
##  $ gpa   : num  0.776 0.81 1 0.534 0.385 ...
##  $ rank  : num  0.667 0.667 0 1 1 ...
```

3 Training and evaluating the model

To train the model, we split the data into a training and test dataset as usual. In this case, the data is split into the predictors (`train.x` and `test.x`) and the outcome (`train.y` and `test.y`) for easier input into `knn()`. We see that the $k = 1$ achieves the best accuracy.

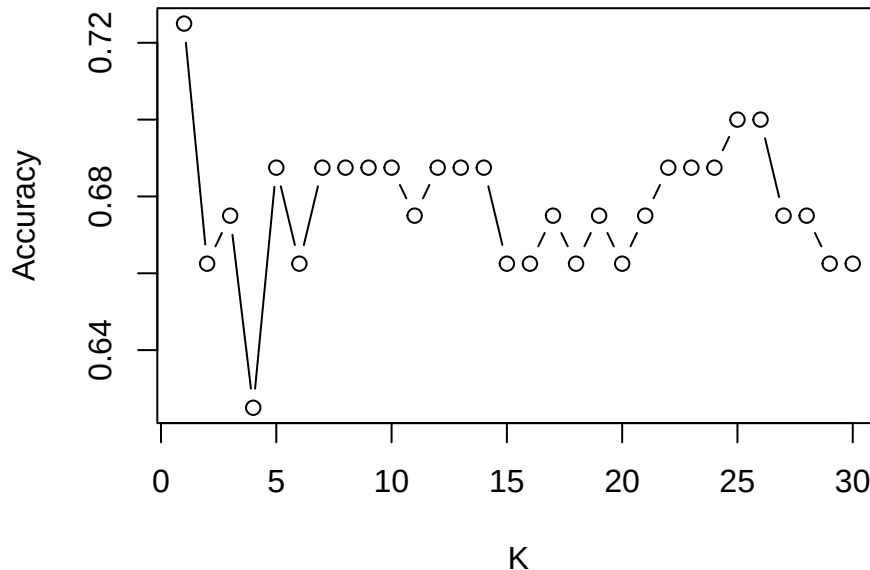
```
set.seed(100)
train.idx <- sample(nrow(grad.nor), nrow(grad.nor)*0.8)
train.x <- grad.nor[train.idx, 2:4]
train.y <- grad.nor[train.idx, 1]
test.x <- grad.nor[-train.idx, 2:4]
test.y <- grad.nor[-train.idx, 1]

library(class)
accuracy <- rep(0, 30)
for (i in 1:30) {
  set.seed(101)
  knn.i <- knn(train.x, test.x, cl=train.y, k=i)
  accuracy[i] <- mean(knn.i == test.y)
  cat("k =", i, "accuracy =", accuracy[i], "\n")
}
```

```
## k = 1 accuracy = 0.725
## k = 2 accuracy = 0.6625
## k = 3 accuracy = 0.675
## k = 4 accuracy = 0.625
## k = 5 accuracy = 0.6875
## k = 6 accuracy = 0.6625
## k = 7 accuracy = 0.6875
## k = 8 accuracy = 0.6875
## k = 9 accuracy = 0.6875
## k = 10 accuracy = 0.6875
## k = 11 accuracy = 0.675
## k = 12 accuracy = 0.6875
## k = 13 accuracy = 0.6875
## k = 14 accuracy = 0.6875
## k = 15 accuracy = 0.6625
```

```
## k = 16 accuracy = 0.6625
## k = 17 accuracy = 0.675
## k = 18 accuracy = 0.6625
## k = 19 accuracy = 0.675
## k = 20 accuracy = 0.6625
## k = 21 accuracy = 0.675
## k = 22 accuracy = 0.6875
## k = 23 accuracy = 0.6875
## k = 24 accuracy = 0.6875
## k = 25 accuracy = 0.7
## k = 26 accuracy = 0.7
## k = 27 accuracy = 0.675
## k = 28 accuracy = 0.675
## k = 29 accuracy = 0.6625
## k = 30 accuracy = 0.6625
```

```
plot(accuracy, type="b", xlab="K", ylab="Accuracy")
```



```
which.max(accuracy)
```

```
## [1] 1
```

Evaluating this model with the confusion matrix, we see this kNN model has an accuracy of 72.5%, a false negative rate of 40% (10/25), and a false positive rate of 21.8% (12/55).

```
set.seed(101)
yhat.knn <- knn(train.x, test.x, cl=train.y, k=1)
mean(yhat.knn == test.y)
```

```
## [1] 0.725
```

```
table(predicted=yhat.knn, observed=test.y)
```

```
##           observed
## predicted  0    1
##           0 43 10
##           1 12 15
```

4 Logistic regression model

A logistic regression model is trained and evaluated on the same data. There will be some changes, however: we will treat rank as a categorical variable, and for interpretability, we will use the original unscaled predictors.

```
grad <- read.csv('gradadmit.csv', header=T)
grad$admit <- factor(grad$admit)
grad$rank <- factor(grad$rank)
str(grad)
```

```
## 'data.frame':    400 obs. of  4 variables:
## $ admit: Factor w/ 2 levels "0","1": 1 2 2 2 1 2 2 1 2 1 ...
## $ gre  : int  380 660 800 640 520 760 560 400 540 700 ...
## $ gpa  : num  3.61 3.67 4 3.19 2.93 3 2.98 3.08 3.39 3.92 ...
## $ rank : Factor w/ 4 levels "1","2","3","4": 3 3 1 4 4 2 1 2 3 2 ...
```

```
train.data <- grad[train.idx,]
test.data <- grad[-train.idx,]
```

The training process goes as usual. The model we get is:

$$\log(\text{odds}) = -3.759 + 0.0019 \times \text{gre} + 0.847 \times \text{gpa} - 0.893 \times \text{rank2} - 1.479 \times \text{rank3} - 1.800 \times \text{rank4}$$

We represent observations with rank 2 with $\text{rank2} = 1$ and $\text{rank3} = \text{rank4} = 0$, and likewise for rank 3 and rank 4. How, then, are rank 1 observations represented?

Observations with rank 1 are represented with $\text{rank2} = \text{rank3} = \text{rank4} = 0$, and the changes in log-odds (coefficients) are then expressed with respect to rank 1.

For example, going from rank 1 to rank 4 represents a change in log odds of -1.8. Going from rank 1 to rank 3 represents a change in log odds of -1.479.

To express the change from rank 2 to rank 3, simply consider the difference when $\text{rank2} = 1$ and $\text{rank3} = 1$. The change in log odds going from rank 2 to rank 3 is then $-1.479 - (-0.893) = -0.586$.

The logistic model achieves an accuracy of 68.8%, a false negative rate of 76%, and a false positive rate of 10.9%. How can you bring down this false negative rate? Will there be any trade-offs?

```
m.logistic <- glm(admit ~ ., train.data, family="binomial")
summary(m.logistic)
```

```
##
## Call:
## glm(formula = admit ~ ., family = "binomial", data = train.data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -3.759335    1.251839  -3.003 0.002673 **
## gre          0.001926    0.001231   1.565 0.117602
## gpa          0.847919    0.364026   2.329 0.019844 *
## rank2       -0.893725    0.355100  -2.517 0.011842 *
## rank3       -1.479118    0.386290  -3.829 0.000129 ***
## rank4       -1.800614    0.474230  -3.797 0.000147 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 400.59  on 319  degrees of freedom
## Residual deviance: 365.34  on 314  degrees of freedom
## AIC: 377.34
##
## Number of Fisher Scoring iterations: 4
```

```
y <- test.data$admit
yhat.p <- predict(m.logistic, test.data, type="response")
yhat <- factor(ifelse(yhat.p > 0.5, 1, 0))
mean(yhat == y)
```

```
## [1] 0.675
```

```
table(predicted=yhat, observed=y)
```

```
##           observed
## predicted  0    1
##           0 48 19
##           1  7  6
```

4.1 Logistic regression with scaled data

Now, we consider the same logistic regression model with scaled data. The model has significantly different coefficients for the gre and gpa variables: this is because a one-unit change is on the scaled data!

$$\log(\text{odds}) = -1.419 + 1.117 \times \text{gre} + 1.475 \times \text{gpa} + \dots$$

And a one-unit change on the scaled data represents a change of going from the **minimum value** to **maximum value** in the original unscaled data.

For example, a one-unit change in the scaled gre here represents a 580-unit change in the original unscaled data!

```
grad <- read.csv('gradadmit.csv', header=T)
grad$admit <- factor(grad$admit)
grad$rank <- factor(grad$rank)

nor <- function(x) { (x - min(x)) / (max(x) - min(x)) }
grad[2:3] <- sapply(grad[2:3], nor)

train.data <- grad[train.idx,]
test.data <- grad[-train.idx,]

m.logistic <- glm(admit ~ ., train.data, family="binomial")
summary(m.logistic)
```

```
##
## Call:
## glm(formula = admit ~ ., family = "binomial", data = train.data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -1.4194      0.5907  -2.403 0.016259 *
## gre           1.1170      0.7137   1.565 0.117602
## gpa           1.4754      0.6334   2.329 0.019844 *
```

```
## rank2          -0.8937      0.3551  -2.517  0.011842  *
## rank3          -1.4791      0.3863  -3.829  0.000129  ***
## rank4          -1.8006      0.4742  -3.797  0.000147  ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 400.59  on 319  degrees of freedom
## Residual deviance: 365.34  on 314  degrees of freedom
## AIC: 377.34
##
## Number of Fisher Scoring iterations: 4
```

This scaling in the predictors, however, does not affect our final results.

```
yhat.p <- predict(m.logistic, test.data, type="response")
yhat <- factor(ifelse(yhat.p > 0.5, 1, 0))
table(predicted=yhat, observed=y)
```

```
##           observed
## predicted  0  1
##           0 48 19
##           1  7  6
```

4.2 Scaling separately

Some of you may think of scaling the training data and testing data separately, i.e.:

```
grad <- read.csv('gradadmit.csv', header=T)
grad$admit <- factor(grad$admit)
grad$rank <- factor(grad$rank)

nor <- function(x) { (x - min(x)) / (max(x) - min(x)) }

train.data <- grad[train.idx,]
test.data <- grad[-train.idx,]

summary(train.data[2:3])
```

```
##           gre           gpa
## Min.      :220.0   Min.    :2.260
## 1st Qu.:500.0   1st Qu.:3.127
```



```
## Median :580.0 Median :3.390
## Mean :584.1 Mean :3.387
## 3rd Qu.:660.0 3rd Qu.:3.652
## Max. :800.0 Max. :4.000
```

```
summary(test.data[,2:3])
```

```
##      gre      gpa
## Min.   :300.0 Min.   :2.480
## 1st Qu.:520.0 1st Qu.:3.138
## Median :610.0 Median :3.400
## Mean   :602.2 Mean   :3.402
## 3rd Qu.:700.0 3rd Qu.:3.675
## Max.   :800.0 Max.   :4.000
```

```
train.data[,2:3] <- sapply(train.data[,2:3], nor)
test.data[,2:3] <- sapply(test.data[,2:3], nor)
summary(train.data[,2:3])
```

```
##      gre      gpa
## Min.   :0.0000 Min.   :0.0000
## 1st Qu.:0.4828 1st Qu.:0.4986
## Median :0.6207 Median :0.6494
## Mean   :0.6277 Mean   :0.6477
## 3rd Qu.:0.7586 3rd Qu.:0.8003
## Max.   :1.0000 Max.   :1.0000
```

```
summary(test.data[,2:3])
```

```
##      gre      gpa
## Min.   :0.0000 Min.   :0.0000
## 1st Qu.:0.4400 1st Qu.:0.4326
## Median :0.6200 Median :0.6053
## Mean   :0.6045 Mean   :0.6063
## 3rd Qu.:0.8000 3rd Qu.:0.7862
## Max.   :1.0000 Max.   :1.0000
```

However, this is extremely problematic, as the data **no longer have the same scaling!** The minimum value in the training data was 220, which is represented as 0 in the scaled training data. The minimum value in the test data was 300, which is now represented as 0 in the scaled test data. This is unusable, because now a 0 in the test data represents a 220 in the original data - when it should have represented 300!

My recommendation is for logistic regression to have **no scaling** whatsoever, as it allows for a much more natural interpretation.